

CVM++ Project and Build Guide

From Zero to Running - Setup, Structure, Definitions, and Step-by-Step Construction

CVM++ Documentation

May 2026

Contents

1	About This Document	2
2	1. What Is CVM++?	2
3	2. Prerequisites	2
3.1	2.1 Required software	2
3.2	2.2 Optional (documentation)	2
3.3	2.3 Knowledge assumed	3
4	3. Quick Start (Clone → Build → Run)	3
4.1	3.1 Clone the repository	3
4.2	3.2 Build	3
4.3	3.3 Run a program	3
4.4	3.4 Verify everything works	3
5	4. Repository Structure	3
5.1	4.1 Source modules (<code>src/</code> <code>include/cvm++/</code>)	4
6	5. Step-by-Step: How This Project Was Built	4
6.1	Step 1 — Foundation	4
6.2	Step 2 — Parser and AST	5
6.3	Step 3 — Bytecode and compiler	5
6.4	Step 4 — Virtual machine	5
6.5	Step 5 — Product polish	5
7	6. Build System Details	6
7.1	6.1 Makefile targets	6
7.2	6.2 Compiler flags	6
7.3	6.3 Exit codes (file mode)	6
8	7. CVM++ Language Reference (Short)	6
9	8. Complete Glossary	7
10	9. Examples Catalog	8
11	10. Troubleshooting	8
12	11. Publishing to GitHub (Checklist)	9

13 12. Further Reading

9

1 About This Document

This guide is written for **anyone** opening the CVM++ repository: students, reviewers, interviewers, or future you. It explains **what the project is**, **how to build and run it**, **how the repository is organized**, the **recommended order for building each part**, and a **complete glossary** of technical terms used throughout the codebase.

Companion document: *CVM++ Architecture & Workflow Guide* (PDF) — explains how code flows through the system at runtime.

2 1. What Is CVM++?

CVM++ is a small **scripting language** plus a **toolchain** written in **C++17**:

Piece	Role
Language	.cvm text files with variables, math, logic, functions, if/while , print , input
Lexer	Turns source text into tokens
Parser	Turns tokens into an AST (tree of meaning)
Compiler	Turns AST into bytecode (<code>std::vector<uint8_t></code>)
Virtual Machine (VM)	Executes bytecode on a stack
CLI / REPL	Terminal program cvmpp to run scripts interactively

You are **not** using LLVM, Python, or an off-the-shelf interpreter library. The pipeline is implemented directly in this repository.

3 2. Prerequisites

3.1 2.1 Required software

Tool	Minimum	Purpose
C++ compiler	GCC 9+, Clang 10+, or MSVC 2019+	Build cvmpp
Make	Any	make / make verify
CMake	3.16+	Optional alternative build
Git	Any	Clone and version control

3.2 2.2 Optional (documentation)

Tool	Purpose
Pandoc + LaTeX (TeX Live / MacTeX)	Regenerate PDF guides: <code>./scripts/build-docs.sh</code>

3.3 2.3 Knowledge assumed

- Basic command line (`cd`, `ls`, running executables)
- Very basic C++ (classes, `std::vector`, `std::string`) if you plan to modify the code

No prior compiler-course background is required to **run** the project.

4 3. Quick Start (Clone → Build → Run)

4.1 3.1 Clone the repository

```
git clone https://github.com/YOUR_USERNAME/cvmpp.git
cd cvmpp
```

Replace YOUR_USERNAME with your GitHub username after you publish the repo.

4.2 3.2 Build

Option A — Make (recommended):

```
make
```

Produces: `build/cvmpp`

Option B — CMake:

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build
```

On Windows, use **WSL** or **MSVC** with the same commands inside a developer shell.

4.3 3.3 Run a program

```
./build/cvmpp examples/hello.cvm
./build/cvmpp -d examples/functions.cvm    # debug: tokens, AST, bytecode
./build/cvmpp -q examples/factorial.cvm    # quiet output only
./build/cvmpp                             # interactive REPL
```

4.4 3.4 Verify everything works

```
make verify
```

Expected: 11 passed, 0 failed.

5 4. Repository Structure

Every top-level path and its purpose:

<code>cvmpp/</code>	
<code>README.md</code>	Project overview (start here on GitHub)
<code>LICENSE</code>	MIT license
<code>CV.md</code>	Resume bullet points (optional for readers)
<code>CONTRIBUTING.md</code>	How to contribute / report issues
<code>Makefile</code>	<code>make</code> , <code>make verify</code> , <code>make clean</code>
<code>CMakeLists.txt</code>	CMake build definition

<code>.gitignore</code>	Files Git should ignore
<code>.github/workflows/ci.yml</code>	GitHub Actions: build + verify
<code>include/cvm++/</code>	Public C++ headers (API of each module)
<code>src/</code>	Implementations (.cpp)
<code>examples/</code>	Sample .cvm programs
<code>scripts/</code>	
<code>verify.sh</code>	Automated example tests
<code>build-docs.sh</code>	Build PDF documentation
<code>docs/</code>	
<code>README.md</code>	Documentation index
<code>guides/</code>	Markdown sources for PDFs
<code>pdf/</code>	Generated PDFs (committed for GitHub)
<code>pandoc/</code>	PDF styling defaults

5.1 4.1 Source modules (`src/` `include/cvm++/`)

File(s)	Module	Responsibility
<code>lexer.cpp / lexer.hpp</code>	Lexer	Characters → tokens
<code>token.cpp / token.hpp</code>	Token	Token types and names
<code>parser.cpp / parser.hpp</code>	Parser	Tokens → AST
<code>ast.cpp, ast.hpp, ast_print.cpp</code>	AST	Tree nodes + debug printer
<code>compiler.cpp / compiler.hpp</code>	Compiler	AST → bytecode
<code>opcode.cpp / opcode.hpp</code>	Opcode	Instruction enum + names
<code>bytecode.cpp / bytecode.hpp</code>	Bytecode	Byte buffer, disassembler
<code>vm.cpp / vm.hpp</code>	VM	Execute bytecode
<code>value.cpp / value.hpp</code>	Value	Runtime <code>int</code> / <code>bool</code>
<code>compile.cpp / compile.hpp</code>	Frontend	Wires <code>lexer</code> → <code>parser</code> → <code>compiler</code> → <code>VM</code>
<code>diagnostic.cpp / diagnostic.hpp</code>	Diagnostics	Errors by phase
<code>ui.cpp / ui.hpp</code>	UI	Colors, tables, help text
<code>main.cpp</code>	CLI	REPL, file runner, flags

The **executable** `cvmpp` links all of the above into one binary.

6 5. Step-by-Step: How This Project Was Built

This section is the **construction roadmap** — the order used to implement a working interpreter. If you rebuild or extend CVM++, follow this sequence.

6.1 Step 1 — Foundation

Goal: Locations, errors, tokens, and a minimal `main`.

1. `source_loc.hpp` — line/column for error messages
2. `diagnostic.hpp` — Phase, Severity, DiagnosticBag
3. `token.hpp` — TokenType, Token
4. `lexer.hpp` — scan identifiers, numbers, operators, keywords
5. `ui.hpp` — colored printing
6. `main.cpp` — read a line, tokenize, print token table

Definition checkpoint: A **token** is one classified chunk of source (e.g. keyword `let`, number `42`).

6.2 Step 2 — Parser and AST

Goal: Validate grammar and build a tree.

1. `ast.hpp` — Expr, Stmt, Program, literals, if, while, let, print
2. `parser.hpp` — recursive-descent parsing
3. `ast_print.cpp` — print tree for debugging

Checkpoint: **AST** = Abstract Syntax Tree; it stores *structure*, not text.

6.3 Step 3 — Bytecode and compiler

Goal: Lower AST to portable bytes.

1. `opcode.hpp` — instruction set (PUSH_INT, ADD, JUMP, ...)
2. `bytecode.hpp` — BytecodeChunk, BytecodeWriter, disassembler
3. `compiler.hpp` — walk AST, emit opcodes, patch jump targets

Checkpoint: **Bytecode** = numeric instructions the VM understands.

6.4 Step 4 — Virtual machine

Goal: Run programs.

1. `value.hpp` — `std::variant<int64_t, bool>`
2. `vm.hpp` — stack, globals, instruction loop, failsafes
3. `compile.hpp` — `compile_frontend()`, `execute()`

Checkpoint: **VM** = program that simulates a CPU for your bytecode.

6.5 Step 5 — Product polish

Goal: Usable tool for demos and portfolio.

1. REPL with `:help`, `:run`, `:disasm`, multiline `{ }`
2. File mode: `cvmpp script.cvm, flags -d -q -c`

3. `examples/*.cvm + scripts/verify.sh`
 4. Functions: `fn`, `return`, `CALL` / `RETURN`, `locals`
 5. CI on GitHub Actions
-

7 6. Build System Details

7.1 6.1 Makefile targets

Target	Command	Effect
<code>all</code> (default)	<code>make</code>	Build <code>build/cvmpp</code>
<code>verify</code>	<code>make verify</code>	Run all example scripts
<code>clean</code>	<code>make clean</code>	Remove binary
<code>repl</code>	<code>make repl</code>	Start REPL
<code>cmake</code>	<code>make cmake</code>	Configure + build via CMake

7.2 6.2 Compiler flags

- `-std=c++17` — language standard
- `-Wall -Wextra -Wpedantic` — strict warnings
- `-Iinclude` — header search path

7.3 6.3 Exit codes (file mode)

Code	Meaning
0	Success
1	Lexer, parser, or compiler error
2	Runtime VM error (e.g. divide by zero)

8 7. CVM++ Language Reference (Short)

```
let x = 10;
x = x + 1;
print x;

if (x < 20) {
  print true;
} else {
  print false;
}

while (x > 0) {
  x = x - 1;
}
```

```
fn add(a, b) {
    return a + b;
}
print add(3, 4);

let name = input;    // reads one line from stdin
```

Construct	Syntax
Integers	0, 42, 1000
Booleans	true, false
Declare	let name = expr;
Assign	name = expr;
Function	fn name(p1, p2) { ... }
Return	return expr;
Call	name(arg1, arg2);
Operators	+ - * / == != < > <= >=
Unary minus	-x
Comment	// to end of line

Not supported: strings, floats, arrays, **for**, classes, modules.

9 8. Complete Glossary

Terms are listed alphabetically. Every term appears somewhere in the project or PDF workflow guide.

Term	Definition
Abstract Syntax Tree (AST)	Tree data structure representing program structure after parsing. Nodes: expressions and statements.
Arity	Number of parameters a function accepts.
Bytecode	Sequence of bytes (<code>std::vector<uint8_t></code>) encoding VM instructions.
BytecodeChunk	Struct holding code, names (variable pool), and functions metadata.
Call frame	VM record of return address + local variables during a function call.
CLI	Command-line interface; running <code>cvmp</code> with a file path.
Compiler	Module that translates AST to bytecode (not a separate executable).
Control flow	<code>if</code> , <code>else</code> , <code>while</code> , and jumps in bytecode.
Diagnostic	Structured error/warning with phase, location, message, hint.
Disassembler	Tool that prints human-readable opcode listing from bytecode.
Front end	Lexer + parser + compiler together (<code>compile_frontend</code>).
Function table	<code>BytecodeChunk::functions</code> — name, entry address, arity.
Global variable	Variable stored outside functions; persists in REPL via <code>VmSession</code> .
Identifier	Name for variable or function (<code>x</code> , <code>factorial</code>).
Instruction pointer (IP)	Index into bytecode array; advances as VM runs.

Term	Definition
ISA	Instruction Set Architecture — list of opcodes the VM supports.
Jump patching	Compiler writes placeholder offset, fills real target after label is known.
Lexer	Scanner converting source text to tokens.
Literal	Fixed value in source: integer or boolean.
Local variable	Slot inside a function (<code>LOAD_LOCAL</code> / <code>STORE_LOCAL</code>).
Name pool	<code>chunk.names</code> — interned variable names referenced by index.
Opcode	One-byte instruction tag (e.g. <code>Add</code> , <code>Call</code>).
Operand	Extra bytes after opcode (e.g. jump offset, name index).
Parser	Builds AST from token stream; reports syntax errors.
Phase	Stage tag on diagnostics: <code>LEXER</code> , <code>PARSER</code> , <code>COMPILER</code> , <code>VM</code> , <code>REPL</code> .
REPL	Read-Eval-Print Loop; interactive <code>cvmp</code> without script file.
Recursive descent	Parsing style: one function per grammar rule.
Runtime	VM execution phase (after compile succeeds).
Source location	Line and column (<code>SourceLoc</code>) for error caret display.
Stack	LIFO structure holding intermediate expression values in VM.
Stack machine	VM design: operations consume/produce stack slots.
Token	Lexeme + type + source range (e.g. <code>Integer "42"</code>).
Type (runtime)	Only <code>int64_t</code> or <code>bool</code> in <code>VmValue</code> variant.
Virtual Machine (VM)	Interpreter loop executing bytecode.
VmSession	REPL-only map keeping global variables between lines.

10 9. Examples Catalog

File	Demonstrates
<code>hello.cvm</code>	<code>let</code> , <code>if</code> , <code>print</code>
<code>arithmetic.cvm</code>	Operators
<code>booleans.cvm</code>	<code>true</code> / <code>false</code>
<code>if_else.cvm</code>	Branches
<code>factorial.cvm</code>	<code>while</code> loop
<code>functions.cvm</code>	Recursive fn / <code>return</code>
<code>comparisons.cvm</code>	<code>!=</code> <code><=</code> <code>>=</code>
<code>assignment.cvm</code>	Reassignment
<code>multiline_demo.cvm</code>	Block summation
<code>input_demo.cvm</code>	<code>stdin</code> input
<code>div_by_zero.cvm</code>	Runtime error (exit 2)

11 10. Troubleshooting

Problem	Solution
<code>cvmp</code> : command not found	Run from repo root; use <code>./build/cvmp</code> or <code>make</code> first

Problem	Solution
Build fails on C++	Install Xcode CLI tools (macOS) or <code>build-essential</code> (Linux)
<code>make verify</code> fails one script	Run <code>./build/cvmpp -d examples/<file>.cvm</code> for diagnostics
PDF build fails	Install <code>pandoc</code> and MacTeX/TeX Live; run <code>./scripts/build-docs.sh</code>
REPL variables “lost”	File runs <code>reset VM</code> ; REPL keeps <code>VmSession</code> globals

12 11. Publishing to GitHub (Checklist)

1. Create empty repo on GitHub (e.g. `cvmpp`).
 2. `git init && git add . && git commit -m "Initial public release"`
 3. `git remote add origin git@github.com:YOUR_USERNAME/cvmpp.git`
 4. `git push -u origin main`
 5. Enable **Actions** tab — CI should pass on push.
 6. Pin **README** and link both PDFs under `docs/pdf/`.
-

13 12. Further Reading

- *Crafting Interpreters* — Robert Nystrom (conceptual background)
 - **Architecture & Workflow Guide (PDF)** — byte-level execution trace in this repo
 - Source entry: `src/compile.cpp` → `compile_frontend()`
-

CVM++ — MIT License — Documentation generated for public repository release.